```
    ui.update(<p>Process complete!</p>);
    ui.done();
  } catch (error) {
    ui.error(error);
  }

  return ui;
}
```

Here we use update again to replace all existing content with a final message.

The done method is called to indicate that no more updates will be made.

If an error occurs, we use the error method to update the UI accordingly.

How does this approach align with building AI web applications? Both helpers allow us to combine the best of both worlds: server-side component access and seamless client-side updates. This ensures sensitive secrets remain secure on the server while reducing the client-side bundle size.

A significant challenge we face now is managing state, particularly sharing message history between the server and client without introducing complex state management solutions. Based on our experience, proper state management is crucial for maintaining a consistent and responsive user experience.

4.2 Managing UI state in AI-powered applications

As our applications grow, simple state management with React's `useState` hook becomes inadequate. We need to handle not only the messages displayed in the UI but also metadata, AI context, and server-side state, all while ensuring smooth performance and real-time updates.

React's `Context` API, with its `useContext` hook, offers a partial solution for propagating variables, but it falls short when we step into the realm of RSCs. In fact, all React hooks do not work in that space. Moreover, we often need server actions to update conversation messages automatically, ensuring the UI reflects changes without manual state updates.

This leads to a dual-state management approach, which separates AI state and UI state. This is a recommended strategy for building robust, responsive AI-powered applications. UI state handles what the user sees and interacts with, while AI state manages the core data and context that our AI models work with behind the scenes.

4.2.1 Separating AI and UI state in React/Next.js applications

When building AI-powered applications with React or Next.js, it's essential to distinguish between two types of state: AI state and UI state. AI state encompasses all the data and context that the AI model needs to function effectively, and UI state deals with the data and state that directly affect what the user sees and interacts with. This separation allows for more efficient handling of data, improved performance, and better synchronization between the server and client.

Figure 4.8 shows the AI and UI state flow in UI applications. First, the user input is received by a client component. The AI context provider manages both UI and AI states. AI state is sent to a server action, which interacts with the LLM/AI model, and the generated content updates both AI and UI states. Then the AI state is serialized and